

Versi 2.0

Januari 2026



PRAKTIKUM SISTEM OPERASI

MODUL 5 - PEMROGRAMAN SHELL DAN OTOMASI SISTEM

(BASH SCRIPTING)

Pengarang:

Ir. Denar Regata Akbi, S.Kom., M.Kom.

Putri Nayla Sabri

Muhammad Zamzam Baihaqi

LABORATORIUM INFORMATIKA

Universitas Muhammadiyah Malang

DAFTAR ISI

DAFTAR ISI.....	2
PENDAHULUAN.....	3
TUJUAN.....	3
TARGET MODUL.....	3
PERSIAPAN.....	3
KATA KUNCI.....	3
LATAR BELAKANG TEORI.....	4
1. Pengantar Pemrograman Shell.....	4 1.1 Apa itu Pemrograman
Shell.....	4 1.2 Mengapa Pemrograman
Shell?.....	4 2. Dasar-Dasar Skrip
Bash.....	4 2.1 Struktur
Skrip.....	4 LATIHAN 1: Dasar Skrip (Struktur &
Variabel).....	6 2.2 Variabel dan Penanganan
Data.....	6 2.3 Input dan
Output.....	8 3. Struktur Kontrol di
Bash.....	10
3.1 Pernyataan Bersyarat.....	10
LATIHAN 2: Ahli Logika (Kondisi).....	11 3.2 Struktur
Perulangan.....	11 LATIHAN 3: Spesialis Perulangan
(Otomasi & Daftar).....	12 3.3 Fungsi
Dasar.....	12
4. Praktik Pemrograman dan Otomatisasi Shell.....	12 4.1 Argumen
Skrip.....	12 4.2 Tugas Otomatisasi
Umum.....	13
LATIHAN 4: Monitor Sistem (Argumen & Fungsi).....	14 4.3 Debugging dan Praktik
Terbaik.....	14 4.4 Skrip Otomatis (Otomasi
Sistem).....	15 LATIHAN 5: Automator (Cron &
Otomasi).....	16 APLIKASI
PRAKTIS.....	17
LATIHAN.....	17
DEMO - STUDI KASUS.....	17
Aturan Demo.....	17
RUBRIK PENILAIAN.....	19
SKALA PENILAIAN.....	19



Bash, Pemrograman Shell, Variabel, Input/Output, Argumen Skrip, Otomatisasi, Debugging

LATAR BELAKANG TEORI

1. Pengantar Pemrograman Shell 1.1 Apa

itu Pemrograman Shell

Pemrograman shell adalah proses menulis serangkaian perintah dalam file teks yang dapat dieksekusi oleh shell sebagai sebuah program. Alih-alih mengetik perintah satu per satu di terminal, skrip shell memungkinkan perintah-perintah ini dikelompokkan dan dieksekusi secara otomatis secara berurutan. Hal ini menjadikan pemrograman shell sebagai cara yang efisien untuk melakukan tugas secara berulang dan konsisten.

Di sebagian besar distribusi Linux, **Bash (Bourne Again Shell)** adalah shell default. Bash bertindak sebagai antarmuka antara pengguna dan sistem operasi, menerjemahkan perintah yang dimasukkan oleh pengguna atau dibaca dari file skrip. Karena Bash tersedia secara luas dan kompatibel dengan shell sebelumnya, Bash telah menjadi pilihan standar untuk scripting shell di lingkungan Linux.

Terdapat perbedaan penting antara **shell interaktif** dan **skrip shell**. Dalam shell interaktif, perintah dieksekusi segera setelah pengguna mengetiknyanya ke terminal.

Sebaliknya, skrip shell berisi perintah-perintah yang telah ditentukan sebelumnya yang tersimpan dalam sebuah file, yang dieksekusi sebagai satu kesatuan saat skrip dijalankan. Perbedaan ini memungkinkan skrip untuk mengotomatisasi tugas tanpa memerlukan interaksi pengguna secara terus-menerus.

1.2 Mengapa Menggunakan Shell Scripting?

Pemrograman shell terutama digunakan untuk **otomatisasi tugas-tugas berulang**. Tugas-tugas seperti membuat file, memeriksa penggunaan disk, mengelola izin, atau mencadangkan direktori sering mengikuti pola yang sama. Menulis skrip memungkinkan tugas-tugas ini diselesaikan secara otomatis, menghemat waktu dan mengurangi kesalahan yang disebabkan oleh input manual.

Dalam **administrasi sistem**, skrip shell banyak digunakan untuk mengelola pengguna, mengkonfigurasi izin file, memantau sumber daya sistem, dan memproses file log. Skrip membantu administrator menerapkan operasi yang konsisten di seluruh sistem dan dengan cepat menanggapi kebutuhan pemeliharaan rutin.

Pemrograman shell sangat terkait dengan perintah shell biasa. Pada dasarnya, skrip shell adalah kumpulan perintah yang dapat dieksekusi secara manual di terminal. Perbedaannya adalah skrip mengatur perintah-perintah ini ke dalam bentuk yang dapat digunakan kembali dan otomatis. Hal ini menjadikan pemrograman shell sebagai perluasan alami dari keterampilan dasar baris perintah.

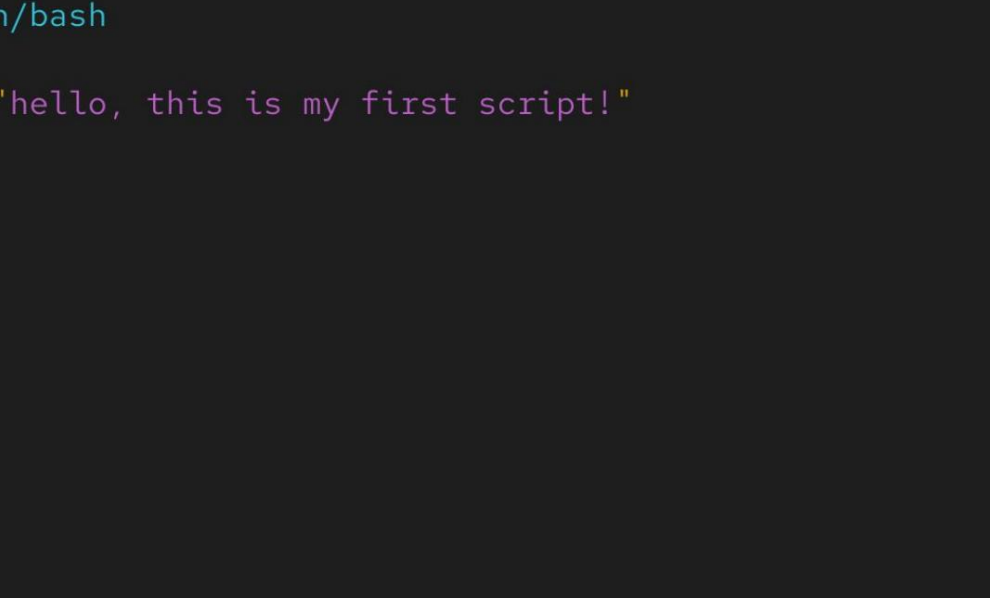
2. Dasar-Dasar Skrip Bash

2.1 Struktur Skrip Skrip

Bash adalah file teks yang berisi serangkaian perintah yang dieksekusi oleh shell. Dasar dari setiap skrip ditentukan oleh bagaimana skrip tersebut diidentifikasi dan diotorisasi oleh sistem.



```
#!/bin/bash
```



```
#!/bin/bash

echo "hello, this is my first script!"

~

~

~

~

~

~

~

~

~

"script.sh" 3L, 52B 3,38 All
```

```
praktikumso@localhost:~/Documents$ chmod +x script.sh
praktikumso@localhost:~/Documents$ ls -l script.sh
-rwxr-xr-x. 1 praktikumso praktikumso 52 Jan 17 14:08 script.sh
```

D. Menjalankan Skrip

Setelah dapat dieksekusi, skrip dapat dijalankan menggunakan jalur relatif atau absolut.

```
praktikumso@localhost:~/Documents$ ./script.sh
hello, this is my first script!
```

LATIHAN 1: Dasar Skrip (Struktur & Variabel)

1. Gunakan **vim** untuk membuat file bernama **hello.sh** dan sertakan baris **shebang** yang benar di bagian paling atas.
2. Deklarasikan variabel bernama **GREETING** dengan nilai "Hello World" (ingat: tidak ada spasi di sekitar tanda =).
3. Gunakan perintah **echo** untuk mencetak nilai variabel Anda dengan menggunakan tanda \$.
4. Simpan file tersebut, lalu gunakan perintah **chmod** untuk membuatnya **dapat dieksekusi**.
5. Jalankan skrip Anda menggunakan sintaks **./hello.sh**.

2.2 Variabel dan Penanganan Data Variabel di

Bash memungkinkan Anda untuk menyimpan dan memanipulasi data secara efisien dalam logika otomatisasi Anda.

A. Deklarasi Variabel

Di Bash, variabel dibuat secara langsung tanpa menggunakan kata kunci khusus.

```
#!/bin/bash
#
name="Raven"
age=19
```

Aturan Penting: •

• **Tanpa Spasi:** Tidak boleh ada spasi sebelum atau sesudah tanda =. • **Penamaan:** Nama variabel tidak boleh diawali dengan angka. • **Peka Huruf Besar-Kecil:**

Variabel peka huruf besar-kecil (misalnya, **Name** berbeda dengan **name**). • **Tanda Kutip:** Sebaiknya gunakan tanda kutip untuk nilai string.

B. Mengakses Nilai Variabel Untuk

mengambil atau menggunakan nilai yang tersimpan dalam variabel, gunakan tanda dolar (\$).



```
#!/bin/bash
```

```
#
```

```
name="Raven"
```

```
age=19
```

```
echo $name
```

```
echo $age
```

Jalankan skrip dengan menjalankan perintah:

```
#!/bash %
```

```
praktikumso@localhost:~/Documents$ vi contoh.sh
```

```
Raven
```

```
19
```

```
Press ENTER or type command to continue
```

C. Variabel Sistem (Lingkungan)

Bash menyediakan variabel bawaan yang menyimpan informasi sistem dan sering digunakan dalam otomatisasi:

Variabel bawaan	Fungsi
\$HOME	Menampilkan direktori beranda pengguna saat ini.
\$JALAN	Menunjukkan lokasi tempat sistem mencari perintah.
\$USER	Menampilkan nama pengguna aktif.

Variabel-variabel ini sangat penting untuk **otomatisasi sistem**, **pembuatan skrip multi-pengguna**, dan **lingkungan konfigurasi**.

D. Operasi Aritmatika

Secara default, Bash memperlakukan variabel sebagai string. Untuk melakukan perhitungan matematika, Anda harus menggunakan Ekspansi aritmatika dengan sintaks `$(())`.



```

praktikumso@localhost:~/Documents - /usr/bin/vim aritmatika.sh
#!/bin/bash

a=10
b=5
echo $((a+b)) #addition
echo $((a-b)) #subtraction
echo $((a*b)) #multiplication
echo $((a/b)) #division
~
~
~
~
~
-- INSERT --                               8,24          All

```

Catatan: Tanpa `$(( ))`, Bash akan memperlakukan operasi tersebut sebagai string biasa dan bukan sebagai perhitungan.

E. Array

Array memungkinkan Anda untuk menyimpan lebih dari satu nilai dalam satu nama variabel.

```

fruits=("apple" "orange" "mango")

echo ${fruits[0]}
~

```

Keluaran:

```

praktikumso@localhost:~/Documents$ vi arrays.sh

apple

Press ENTER or type command to continue

```

2.3 Input dan Output A.

Membaca Input dan Menampilkannya

Skrip Bash dapat berinteraksi dengan pengguna dengan membaca input dan menampilkan output, sehingga membuatnya dinamis dan responsif. Perintah **echo** umumnya digunakan untuk menampilkan pesan atau nilai variabel ke terminal, sedangkan perintah **read** menghentikan sementara eksekusi skrip dan menunggu pengguna mengetikkan informasi.



```
#!/bin/bash

echo "Enter your name: "
read USERNAME
echo "Hello, $USERNAME"

~
```

Saat dijalankan, skrip akan meminta pengguna untuk memasukkan input dan akan menghasilkan output seperti yang ditunjukkan di bawah ini:

```
praktikumso@localhost:~/Documents$ vi inputoutput.sh

Enter your name:
raven
Hello, raven

Press ENTER or type command to continue
```

B. Status Keluar dan Kode Pengembalian

Setiap skrip atau perintah di Linux mengembalikan status keluar, yang merupakan nilai bilangan bulat yang menunjukkan apakah tugas tersebut berhasil atau tidak. Untuk memeriksa status keluar, Linux menyediakan variabel khusus yang disebut **\$?**, Anda dapat memeriksanya dengan menambahkan sintaks ini:

```
echo $?
```

- Sukses (0): Status keluar 0 berarti perintah atau skrip dieksekusi tanpa kesalahan.
- Kesalahan (Bukan nol): Nilai apa pun antara 1 dan 255 menunjukkan kesalahan atau kegagalan tertentu. kondisi.
- Keluar Manual: Anda dapat menentukan status keluar skrip Anda menggunakan perintah **exit** (misalnya, **keluar 0**).

Saat Anda menjalankan skrip, status keluar akan ditampilkan:

```
praktikumso@localhost:~/Documents$ vi inputoutput.sh

Enter your name:
raven
Hello, raven
0

Press ENTER or type command to continue
```



Dalam skrip otomatisasi atau pengembangan modul, status keluar sangat penting untuk pengambilan keputusan logis. Status keluar memungkinkan skrip untuk menangani kesalahan dengan baik; misalnya, jika suatu perintah gagal (status keluar selain 0), skrip dapat diprogram untuk berhenti segera atau mengirimkan pemberitahuan kesalahan sebelum melanjutkan ke langkah berikutnya.

```
#!/bin/bash

mkdir /important_data
if [ $? -ne 0 ]; then
    echo "failed to create directory"
    exit 1
fi

echo "directory created successfully"
```

3. Struktur Kontrol dalam Bash

Struktur kontrol adalah "otak" dari skrip Anda. Struktur ini memungkinkan skrip untuk melampaui sekadar daftar perintah dengan membuat keputusan, mengulang proses, dan mengatur logika ke dalam blok yang dapat digunakan kembali.

3.1 Pernyataan Bersyarat Pernyataan

bersyarat memungkinkan skrip untuk menentukan tindakan apa yang harus dilakukan berdasarkan kriteria tertentu.

A. Pernyataan If

Bentuk logika paling dasar; pernyataan ini hanya menjalankan perintah jika suatu kondisi **benar**.

```
if [condition]; then

fi
```

B. If-else dan Elif

Digunakan ketika terdapat beberapa kemungkinan atau rencana cadangan jika kondisi pertama gagal.

```
if [condition]; then
    #if true
elif [condition]; then
    #if second condition is true
else
    #commands if all conditions are false
fi
```

C. Operator Perbandingan Umum Saat

menulis kondisi di Bash, Anda menggunakan flag khusus di dalam tanda kurung siku []. Perhatikan bahwa spasi diperlukan di dalam tanda kurung.



Kategori	Operator	Keterangan
Numerik	<code>-persamaan</code>	Sama dengan
	<code>-ne</code>	Tidak sama dengan
	<code>-gt</code>	Lebih besar dari
	<code>-lt</code>	Kurang dari
	<code>-ge</code>	Lebih besar dari atau sama dengan
	<code>-le</code>	Kurang dari atau sama dengan
Uji File	<code>-F</code>	Periksa apakah file tersebut ada dan merupakan file biasa.
	<code>-D</code>	Periksa apakah direktori tersebut ada.
	<code>-r/-w/-x</code>	Periksa apakah file tersebut dapat dibaca/ditulis/dieksekusi.
Rangkaian	<code>=</code>	String tersebut sama
	<code>!=</code>	String tidak sama
	<code>-Z</code>	String tersebut kosong (panjangnya nol)
	<code>-N</code>	String tidak kosong

LATIHAN 2: Master Logika (Kondisional)

1. Tulis skrip bernama `check_user.sh` yang meminta pengguna untuk memasukkan nama pengguna.
2. Jika input sesuai dengan nama pengguna Anda saat ini, cetak "Akses Diber granted: Selamat datang" [nama belakang]."
3. Jika inputnya adalah "`root`", cetak "Peringatan: Akses root terdeteksi."
4. Jika tidak, tampilkan "Akses Ditolak: Pengguna tidak dikenali."

3.2 Struktur Perulangan

Perulangan memungkinkan Anda menjalankan perintah berulang kali, menghemat waktu dan mengurangi duplikasi kode.

A. Perulangan For

Paling cocok untuk jumlah iterasi yang diketahui, seperti memproses daftar pengguna atau file.

B. Perulangan While

Berlangsung selama suatu kondisi tetap **benar**.

C. Hingga Loop

Berjalan hingga suatu kondisi menjadi **benar** (kebalikan dari while).



LATIHAN 3: Spesialis Loop (Otomasi & Daftar)

Berlatihlah menggunakan struktur perulangan untuk melakukan tugas berulang secara otomatis.

1. Buat skrip bernama `bulk_create.sh`.
2. Gunakan perulangan `for` untuk membuat 5 direktori bernama `Project_1` hingga `Project_5`.
3. Di dalam setiap direktori, buat secara otomatis file kosong bernama `notes.txt`.
4. Setelah menjalankan skrip, gunakan perintah `tree` atau `ls -R` untuk memverifikasi strukturnya.

3.3 Fungsi Dasar

Fungsi mengelompokkan perintah bersama-sama ke dalam blok bernama, sehingga membuat skrip menjadi modular dan dapat digunakan kembali.

4. Praktik Pembuatan Skrip Shell dan Otomatisasi

Sekarang kita akan fokus pada penerapan skrip Bash untuk skenario dunia nyata, seperti mengotomatisasi tugas sistem, mengelola file secara efisien, dan menulis skrip yang aman dan mudah dipelihara.

4.1 Argumen Skrip Argumen

skrip memungkinkan pengguna untuk memberikan input secara langsung saat skrip dijalankan, sehingga menghilangkan kebutuhan akan perintah `read` interaktif selama eksekusi.

Bash menyediakan serangkaian variabel khusus yang dikenal sebagai *parameter positional* yang memungkinkan skrip menerima input langsung dari baris perintah saat dieksekusi. Variabel-variabel ini membuat skrip lebih fleksibel karena perilakunya dapat berubah tergantung pada argumen yang diberikan oleh baris perintah.

pengguna.

Variabel `$0` mewakili nama skrip itu sendiri, yang berguna untuk menampilkan informasi penggunaan atau pesan kesalahan. Argumen yang diberikan setelah nama skrip disimpan secara berurutan di `$1`, `$2`, `$3`, dan seterusnya, di mana `$1` menyimpan argumen pertama, `$2` argumen kedua, dan seterusnya. Ketika sebuah skrip perlu mengakses semua argumen yang diberikan sekaligus, `$@` dapat digunakan, sedangkan `$#` tersedia untuk menunjukkan berapa banyak argumen yang diberikan secara total.

Misalnya, ketika perintah `./backup.sh /home/praktikumso` dieksekusi, jalur direktori `/home/praktikumso` disimpan dalam `$1`. Di dalam skrip, perintah

```
echo "Target Directory : $1"
```

akan menampilkan pesan *Direktori target: /home/praktikumso*.

Mekanisme ini umum digunakan dalam skenario scripting shell praktis. Skrip pencadangan sering kali bergantung pada argumen untuk menentukan direktori mana yang harus dicadangkan. Dalam administrasi sistem, argumen memungkinkan skrip seperti `./create_user.sh nayla` untuk membuat pengguna secara dinamis.



berdasarkan input. Demikian pula, skrip pemantauan menggunakan argumen untuk menentukan file log, layanan, atau direktori yang perlu diperiksa.

4.2 Tugas Otomatisasi Umum

Pemrograman shell adalah alat utama untuk mengotomatisasi tugas-tugas administrasi sistem mendasar. Daripada menjalankan perintah secara manual satu per satu, administrator dapat mengelompokkan operasi terkait ke dalam satu skrip yang berjalan secara konsisten setiap saat.

Beberapa tugas otomatisasi umum meliputi:

- **Manajemen file dan direktori**, seperti membuat direktori secara otomatis, menyalin file, memindahkan data antar lokasi, atau menghapus sumber daya yang sudah usang.
- **Manajemen izin dan kepemilikan**, di mana skrip menerapkan `chmod` dan `chown` di beberapa file atau direktori untuk menegakkan kebijakan keamanan yang konsisten.
- **Manajemen pengguna dan grup**, misalnya membuat akun pengguna secara massal atau menetapkan pengguna ke grup tertentu seperti `pengembang`.
- **Pemeriksaan log**, yang meliputi membaca dan menyaring file log untuk mengidentifikasi kesalahan sistem atau potensi masalah keamanan.

Manfaat otomatisasi sangat signifikan. Skrip mengurangi risiko kesalahan manusia selama tugas berulang, menghemat banyak waktu administratif, dan memastikan bahwa konfigurasi sistem tetap konsisten di berbagai mesin dan lingkungan.

Pemrograman shell bukan hanya sekadar rangkaian perintah; ini adalah alat untuk menjaga konsistensi sistem. Berikut adalah skenario dunia nyata dan contoh skrip yang umum digunakan oleh administrator:

A. Otomatisasi Pendaftaran

Pengguna Administrator seringkali perlu membuat banyak akun. Skrip ini mempercepat proses dengan membuat pengguna, mengatur `#!/bin/bash`

```

USERNAME=$1
GROUP=$2
# Create group if it doesn't exist [cite: 701]
groupadd $GROUP 2>/dev/null

# Create user with a home directory and assign to group [cite: 692]
useradd -m -g $GROUP $USERNAME
echo "User $USERNAME has been added to group $GROUP." a home directory, and assigning them to
a group automatically.

```



B. Skrip Pemantauan dan Peringatan

Log digunakan untuk menyaring file log guna mengidentifikasi kesalahan atau aktivitas mencurigakan dengan cepat.

```
#!/bin/bash
# Script to check for "failed" attempts in auth log
LOG_FILE="/var/log/auth.log"
SEARCH_TERM="failed"
echo "Searching for $SEARCH_TERM in $LOG_FILE..."
grep -i "$SEARCH_TERM" $LOG_FILE > failed_attempts.txt
echo "Summary saved to failed_attempts.txt"
```

C. Penegakan Izin Batch

Hal ini memastikan bahwa semua file dalam suatu direktori mengikuti kebijakan keamanan yang seragam.

```
#!/bin/bash
# Enforce permissions: 755 for directories, 644 for files
TARGET_DIR=$1

find $TARGET_DIR -type d -exec chmod 755 {} +
find $TARGET_DIR -type f -exec chmod 644 {} +
echo "Permissions for $TARGET_DIR have been standardized."
```

LATIHAN 4: Monitor Sistem (Argumen & Fungsi)

1. Tulis skrip bernama `sys_report.sh` yang menerima satu argumen: `disk`, `user`, atau `all`.
2. Jika argumennya adalah `disk`, tampilkan penggunaan penyimpanan saat ini.
3. Jika argumennya adalah `pengguna`, tampilkan pengguna yang sedang login dan grupnya.
4. Jika argumennya adalah `"semua"`, lakukan kedua tugas tersebut.
5. Bungkus setiap tugas pelaporan (pemeriksaan disk dan pemeriksaan pengguna) di dalam sebuah fungsi dasar.

4.3 Debugging dan Praktik Terbaik Dalam

pembuatan skrip shell, kesalahan dapat terjadi karena sintaks yang salah, input yang tidak terduga, atau perintah yang gagal. Debugging membantu mengidentifikasi di mana dan mengapa skrip gagal, sementara praktik terbaik memastikan bahwa skrip tersebut andal, aman, dan mudah dipelihara dari waktu ke waktu.

Salah satu cara termudah untuk men-debug skrip Bash adalah dengan mengaktifkan pelacakan eksekusi. Dengan menggunakan opsi `set -x`, Bash akan menampilkan setiap perintah sebelum dieksekusi, beserta variabel-variabelnya yang diperluas. Ini memungkinkan penulis skrip untuk mengikuti alur eksekusi dan mendeteksi logika yang salah atau nilai yang tidak terduga.

Contoh:

```
#!/bin/bash
set -x
NAME="Rocky"
echo "Hello $NAME"
```



Keluaran:

```
praktikumso@localhost:~$ ./coba.sh
+ NAME=Rocky
+ echo 'Hello Rocky'
Hello Rocky
```

Opsi berguna lainnya adalah **set -e**, yang memaksa skrip untuk berhenti segera jika ada perintah yang mengembalikan status keluar bukan nol. Perilaku ini penting dalam tugas otomatisasi di mana melanjutkan setelah kegagalan dapat menyebabkan kerusakan data atau keadaan sistem yang tidak konsisten.

```
#!/bin/bash
set -e
# Perintah yang salah (misal: direktori tidak ada)
cd /folder_yang_tidak_ada
# Perintah di bawah ini tidak akan pernah dijalankan karena set -e
echo "Proses selesai"
```

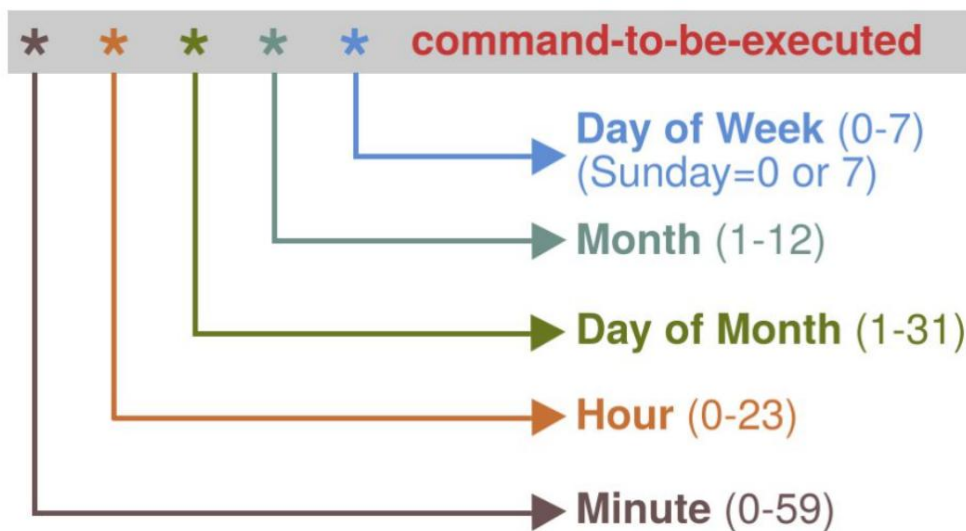
Selain itu, memeriksa status keluar menggunakan **\$?** dapat membantu menentukan apakah suatu perintah berhasil dieksekusi. Hal ini sering digunakan untuk mengambil keputusan atau menampilkan pesan kesalahan yang bermakna.

4.4 Skrip yang Dijalankan Secara Otomatis (Otomasi Sistem)

Agar otomatisasi benar-benar efektif, skrip seringkali perlu dijalankan secara otomatis tanpa intervensi manual. Di Linux, ada beberapa cara untuk mencapai hal ini:

A. Penjadwalan dengan Cron (Tugas Berkala)

Cron adalah daemon penjadwalan tugas berbasis waktu di Linux yang memungkinkan Anda menjalankan skrip secara otomatis pada interval tertentu. Untuk mengelola jadwal ini, pengguna berinteraksi dengan file konfigurasi yang disebut **crontab** (tabel cron) menggunakan perintah **crontab -e**.



Gambar 1. Format Jadwal Crontab



Contoh: Pencadangan Harian pada Tengah Malam

```
0 0 * * * /home/user/scripts/backup.sh
```

Ini akan berjalan pada menit ke-0, jam ke-0, setiap hari dalam bulan, setiap bulan, dan setiap hari dalam minggu.

B. Skrip Startup dengan .bashrc (Tugas Login)

Jika Anda ingin sebuah skrip dijalankan setiap kali pengguna masuk atau membuka terminal

- baru: 1. Buka file .bashrc di direktori home: `vi ~/.bashrc`.
2. Tambahkan jalur skrip di bagian paling bawah: `/path/to/your_script.sh`.
3. Terapkan perubahan dengan menjalankan: `source ~/.bashrc`.

C. Layanan Systemd (Tugas Latar Belakang)

Untuk skrip yang harus berjalan terus menerus di latar belakang sejak sistem dinyalakan: 1. Buat file

- unit layanan di `/etc/systemd/system/myscript.service`.
2. Gunakan perintah `systemctl enable myscript.service` untuk memulainya secara otomatis saat booting.

LATIHAN 5: Automator (Cron & Otomatisasi)

1. Tambahkan perintah `set -x` di awal skrip apa pun untuk mengaktifkan **pelacakan eksekusi** (mode debugging).
2. Buka editor `crontab` menggunakan perintah `crontab -e`.
3. Tambahkan baris untuk menjadwalkan skrip agar berjalan **setiap menit** (menggunakan format `* * * * *`).
4. Verifikasi otomatisasi dengan memeriksa apakah output skrip (seperti file yang dibuat atau entri log) Sesuai harapan.



APLIKASI PRAKTIS

PRAKTIK

Tujuan: Siswa secara langsung mencoba perintah dasar untuk navigasi dan manipulasi file sederhana.

(Latihan belum ditampilkan; siswa menunggu instruksi selama sesi latihan.)

DEMO - STUDI KASUS

Situasi Anda

adalah Administrator Sistem Senior di pusat data perusahaan. Tim mengelola ratusan server Rocky Linux di mana pengaturan lingkungan (membuat ruang kerja proyek, log, dan jejak audit) saat ini dilakukan secara manual. Proses manual ini tidak lagi dapat diskalakan untuk penerapan arsitektur layanan mikro skala besar yang akan datang.

Masalah yang perlu

diidentifikasi adalah kurangnya otomatisasi yang menyebabkan struktur direktori yang tidak konsisten di berbagai server. Upaya pembuatan skrip yang ada gagal karena tidak menangani banyak input, mengabaikan izin sistem (UID/GID), dan tidak menyediakan jejak audit dari apa yang telah dibuat. Selain itu, jika skrip gagal di tengah eksekusi (misalnya, karena disk penuh), skrip tersebut tidak memberikan status keluar, sehingga alat orkestrasi terpusat tidak dapat mendeteksi kegagalan tersebut.

Misi Anda

adalah mengembangkan **Skrip Otomasi** yang tangguh yang: 1.

- Menangani Banyak Argumen:** Menerima sejumlah nama direktori yang tidak terbatas sebagai argumen baris perintah menggunakan perulangan.
- Validasi Input & Izin:** Memeriksa apakah setiap input kosong dan memverifikasi apakah saat ini Pengguna memiliki wewenang untuk membuat direktori di jalur target.
- Pengaturan Ruang Kerja Otomatis:** Untuk setiap direktori valid yang dibuat, skrip harus Secara otomatis menghasilkan file **audit.log** di dalamnya, yang berisi stempel waktu pembuatan dan ID pengguna yang memulai proses tersebut.
- Status Keluar & Debugging:** Implementasi Kode Keluar yang tepat (0 untuk sukses, bukan nol untuk gagal) sehingga sistem dapat melacak hasil otomatisasi.
- Eksekusi Langsung:** Skrip harus dibuat dapat dieksekusi dan dijalankan langsung melalui jalurnya **(./)** untuk menunjukkan penanganan lingkungan yang tepat.

Aturan Demo

- 1. Dilarang Mengakses Modul atau Catatan** Mahasiswa dilarang keras membuka PDF modul, catatan pribadi, atau dokumentasi online apa pun selama sesi demo.
- 2. Penulisan Skrip Hanya dengan Vim** Seluruh proses penulisan dan pengeditan skrip harus dilakukan menggunakan editor teks **Vim**. Penggunaan editor GUI atau editor teks lainnya tidak diizinkan.



3. **Mendemonstrasikan Keterampilan Inti Bash** Siswa harus secara jelas menunjukkan kemahiran dalam:

- Struktur Skrip •
- Penguasaan Eksekusi •
- Variabel & Input •
- Logika & Otomatisasi •
- Input/Output & Penanganan Kesalahan

4. **Verifikasi Akhir** Setelah menjalankan skrip, siswa harus memverifikasi hasil otomatisasi. 5. **Penilaian Konseptual Langsung** Asisten akan melakukan sesi tanya jawab untuk mengevaluasi **Pemahaman Konseptual Anda (40%)**. Bersiaplah untuk menjelaskan logika di balik setiap baris kode yang Anda tulis, karena ini sangat memengaruhi nilai akhir Anda.

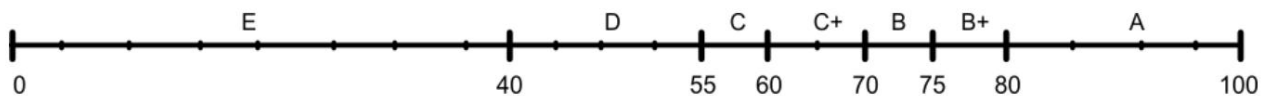
6. **Ikuti Instruksi Langsung** Semua tindakan harus dilakukan secara interaktif mengikuti arahan instruktur atau asisten laboratorium.



RUBRIK PENILAIAN

Aspek Penilaian	Poin (Total 100%)
LATIHAN	Total 5%
Latihan 1	1%
Latihan 2	1%
Latihan 3	1%
Latihan 4	1%
Latihan 5	1%
PRAKTIK	Total 20%
Latihan 1	10%
Latihan 2	10%
DEMO	Total 75%
Penyelesaian Studi Kasus	35%
Pemrograman Bash Dasar	15%
Penguasaan Variabel & Logika	15%
Penguasaan Eksekusi & Izin	10%

SKALA PENILAIAN



A = (81 - 100) ÿ Sepuh

B+ = (75 - 80) ÿ Sangat Baik

B = (70 - 74) ÿ Baik

C+ = (60 - 69) ÿ Cukup Baik

C = (55 - 59) ÿ Adil

D = (41 - 54) ÿ Miskin

E = (0 - 40) ÿ Bro sungguh...

